Pen:240840116014

Practical 1:Study the Data Analytics lifecycle and perform data acquisition from CSV, Excel, and JSON files using Pandas.

Step 1:Import pandas

```
import pandas as pd
```

Step 2:Create the Dataset

```
# Create a dictionary containing student data
data = {
    "ID": [101, 102, 103, 104,105,106,107],
    "Name" :["Rahul", "Priya", "Amit","Riya", "karan", "Krisha", "Harry"],
    "Age": [20, 21, 19,20, 22, 24,28],
    "City": ["Surat", "navsari", "Valsad", "Vyara", "Bardoli","Surat", "Nandurbar"],
    "Marks":[86,85,90,70,60,50,79],
}
```

Step 3:: Create and Display DataFrame

```
 #Create a DataFrame from the given data
df = pd.DataFrame(data)

# Display the DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
2	103	Amit	19	Valsad	90
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60
5	106	Krisha	24	Surat	50
6	107	Harry	28	Nandurbar	79

Step 4: Create CSV file

```
#Create and save the DataFrame as a CSV file
df.to_csv("students.csv", index=False)
#Display confirmation message
print("CSV file created successfully.")
```

```
CSV file created successfully.
```

Step 5: Create Excel file

```
#Create and save the DataFrame as an Excel file
df.to_excel("students.xlsx", index=False)
# Display confirmation message
print("Excel file created successfully.")
```

```
Excel file created successfully.
```

Step 6: Create JSON file

```
#Create and save the DataFrame as a JSON file
df.to_json("students.json", orient="records", indent=4)
# Display confirmation message
print("JSON file created successfully.")
```

```
JSON file created successfully.
```

Step 7: Read CSV file

```
#Read the CSV file using Pandas
df_csv = pd.read_csv("students.csv")
# Display the CSV data
df_csv
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
2	103	Amit	19	Valsad	90
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60
5	106	Krisha	24	Surat	50
6	107	Harry	28	Nandurbar	79

Step 8: Read Excel file

```
#Read the Excel file using Pandas
df_excel = pd.read_excel("students.xlsx")
# Display the Excel data
df_excel
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
2	103	Amit	19	Valsad	90
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60
5	106	Krisha	24	Surat	50
6	107	Harry	28	Nandurbar	79

Step 9: Read JSON file

```
#Read the JSON file using Pandas
df_json = pd.read_json("students.json")
# Display the JSON data
df_json
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
2	103	Amit	19	Valsad	90
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60
5	106	Krisha	24	Surat	50
6	107	Harry	28	Nandurbar	79

Step 10: Display first 5 Data

```
#Display the first 5 rows of the CSV data
df_csv.head()
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
2	103	Amit	19	Valsad	90
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60

Step 11: Display last 5 data

```
#Display the last 5 rows of the CSV data
df_csv.tail()
```

	ID	Name	Age	City	Marks
2	103	Amit	19	Valsad	90
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60
5	106	Krisha	24	Surat	50
6	107	Harry	28	Nandurbar	79

Step 12: Display information of Dataset

```
 #Display information about the dataset
df_csv.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   ID      7 non-null      int64 
 1   Name    7 non-null      object
 2   Age     7 non-null      int64 
 3   City    7 non-null      object
 4   Marks   7 non-null      int64 
dtypes: int64(3), object(2)
memory usage: 412.0+ bytes
```

Step 13: Display Statistical Summary

```
# Generate a summary of the numerical data
df_csv.describe()
```

	ID	Age	Marks
count	7.000000	7.000000	7.000000
mean	104.000000	22.000000	74.285714
std	2.160247	3.109126	14.907652
min	101.000000	19.000000	50.000000
25%	102.500000	20.000000	65.000000
50%	104.000000	21.000000	79.000000
75%	105.500000	23.000000	85.500000
max	107.000000	28.000000	90.000000

step 14: Display Dataset Shape

```
#Check the size of the dataset
df_csv.shape
```

```
(7, 5)
```

Step 15: Display Coloumn Names

```
# View the names of all columns in the dataset
df_csv.columns
```

```
Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

Step 16: Display Data Types

```
#Check the data type
df_csv.dtypes
```

	0
ID	int64
Name	object
Age	int64
City	object
Marks	int64

dtype: object

Step 17: Check for Missing Values

```
#Check for missing values in each column
df_csv.isnull().sum()
```

	0
ID	0
Name	0
Age	0
City	0
Marks	0

dtype: int64

Step 18: Display a single column

```
#Select and display the Marks column from the dataset
df_csv["Marks"]
```

	Marks
0	86
1	85
2	90
3	70
4	60
5	50
6	79

dtype: int64

Step 19:Display Multiple Columns

```
#Select and display the Name and Marks columns
df_csv[["Name", "Marks"]]
```

	Name	Marks
0	Rahul	86
1	Priya	85
2	Amit	90
3	Riya	70
4	karan	60
5	Krisha	50
6	Harry	79

Step20: Filter Records

```
#Filter and display records where marks are greater than 80
df_csv[df_csv["Marks"] > 80]
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
2	103	Amit	19	Valsad	90

Step 21: Calculate Average Marks

```
#Find the average value of the Marks column
df_csv["Marks"].mean()
```

```
np.float64(74.28571428571429)
```

Step 22:Sort Data

```
#Arrange the records according to their marks
df_csv.sort_values(by="Marks", ascending=False)
```

	ID	Name	Age	City	Marks
2	103	Amit	19	Valsad	90
0	101	Rahul	20	Surat	86
1	102	Priya	21	navsari	85
6	107	Harry	28	Nandurbar	79
3	104	Riya	20	Vyara	70
4	105	karan	22	Bardoli	60
5	106	Krisha	24	Surat	50